

Week 01: A Linked List Inventory of Shapes

Focus

In this lab exercise, we will focus on

- basic object oriented programming concepts like classes, objects, instantiation, inheritance, virtual functions, abstract classes, etc and
- basic singly linked list implementation including creation of the list, addition of elements, deletion, and traversal.

Given

You will be given a `givenclasses.h` header file. This file contains several class declarations and some function definitions. Carefully go through the header file to see what classes are given, which ones are inherited, etc. The file will be explained in class. This file should NOT be modified.

Requirement

You must write a single c++ program file (`<rollno>.cpp`) that uses (i.e., includes) the given header file. Your program must be able to read instructions from the console and print output to the console. It should be designed to maintain an inventory (i.e., a list) of polygons. The list should be in the form of a singly linked list.

In our case, we limit ourselves to right triangles, rectangles and squares. As you can see from the header file, they all must derive from the class Polygon. Note that the class Square derives from the class Rectangle.

All the required classes and class functions are declared in the header file. You must provide their complete definitions in your

The input to your program must be fed in from the console line by line. Each line of the input must start with one of the following commands.

- print-all
 - this does not take any further inputs. It will just print all polygons in the inventory. The format should be similar to the way input is collected. See “add” below.
- print
 - This will take one more input: “area” or “circumference”. It will then print the total area or the total circumference of all the polygons in the inventory.
- add
 - This will require several inputs. Firstly, you must specify one of the following (and the subsequent inputs):
 - “right-triangle” followed by
 - a single word representing the name of the object,
 - the base length (double), and
 - the height (also double);
 - “rectangle” followed by
 - a single word representing the name of the object,
 - the base length (double), and
 - the height (also double);
 - “square” followed by
 - a single word representing the name of the object and
 - the side length (double);
 - You should normally be able to add each polygon by specifying the required input in one line. For example, “add right-triangle t2 34.565 532.33” should add a right triangle called t2 with the appropriate dimensions.

- The polygons must be maintained in a singly linked list. The add operation must only take $O(1)$ time, which means that you must add it to the beginning of the list.
- delete
 - This will require one additional input — the name of the polygon you wish to delete.
 - E.g., “delete t2” should delete the polygon with the name t2.
 - If the specified polygon (i.e., t2 in the example) does not exist, do nothing.
 - If there are two or more polygons with the same name, you only need to delete the first occurrence in the linked list.
- done
 - The program must exit gracefully when this command is encountered.

For example, consider the following input.

```
add right-triangle t1 3 4
add square s1 4
add rectangle r1 5 7
add right-triangle t2 5 6
add square s2 6
delete t2
print circumference
print area
done
```

The output must be 76 93.